Reward-Based Intelligence

Intelligence as Reward-Based Learning

Many intelligent behaviours arise from learning through rewards.

Examples:

- Animals learn where food is located
- Children learn which behaviours are rewarded
- People learn strategies in games

Good decisions tend to produce rewards.

Over time, intelligent agents learn to choose actions that produce higher rewards.

Example: Learning by Trial and Error

Consider a mouse in a maze.

The mouse explores the maze and occasionally finds food.

- Paths leading to food are rewarding
- Paths leading to dead ends are not

This is learning through trial and error
See: Claude Shannon and Theseus

Another Example: Learning to Play a Game

Suppose an AI learns to play a video game.

Possible rewards:

- +10 for winning
- -10 for losing
- small rewards for progress

The agent experiments with different actions.

Actions that lead to higher scores are reinforced.
See: AlphaGo: The Movie

The Agent–Environment Interaction

We can describe reward-based learning using a simple loop.

1. Agent observes the current situation
2. Agent chooses an action
3. Environment responds with:
   - a new situation
   - a reward

$$\text{Agent} \rightarrow \text{Action} \rightarrow \text{Environment}$$
$$\text{Environment} \rightarrow \text{State} + \text{Reward} \rightarrow \text{Agent}$$

Reinforcement Learning

Reinforcement learning studies how an agent can learn to act in order to maximize reward.

The agent must learn:

- which actions lead to good outcomes
- which actions should be avoided

Key challenges:

- delayed rewards
- exploration vs exploitation
- uncertainty about the environment

How can an agent learn a good strategy from experience?

The Agent–Environment Loop

$$\text{Agent} \rightarrow \text{Action} \rightarrow \text{Environment}$$
$$\text{Environment} \rightarrow \text{State} + \text{Reward} \rightarrow \text{Agent}$$

At each time step:

1. Agent observes the current state s
2. Agent chooses an action a
3. Environment returns:
   - next state s'
   - reward r

Markov Decision Processes (MDPs)

Simple mathematical model for RL is as an **MDP**:

$$(S, A, P, R, \gamma)$$

- S = states
- A = actions
- $P(s'|s, a)$ = transition probability
- $R(s, a)$ = reward
- γ = discount factor

Goal:
Find a **policy** $\pi(a|s)$ that maximizes expected reward.

Policies

A **policy** tells the agent what to do.

$$\pi(a \mid s) = \text{action } a \text{ taken in state } s$$

Example:

"If I am in this state, move right."

Policies can be:

- deterministic (rules)
- stochastic (probabilities)

Value Functions

The **value** of a state measures how good it is.

$$V(s) = \text{expected future reward from state } s$$

Value depends on:

- the policy
- future rewards
- the discount factor

Bellman Equation and Q-function

Bellman equation:

$$V(s) = \max_a \sum_{s'} P(s'|s,a) \left[R(s') + \gamma V(s')\right]$$

Define:

$$Q(s,a) = \sum_{s'} P(s'|s,a) \left[R(s') + \gamma V(s')\right]$$

So:

$$V(s) = \max_a Q(s,a)$$

$Q(s,a)$ = expected value of taking action a in state s

$$Q(s,a) = \mathbb{E}[R(s') + \gamma \max_{a'} Q(s',a)]$$

Example: Fully Stochastic 2×2 Grid World

States:

s_1	s_2
s_3	s_4

From any state and action:

- 0.7: go to intended neighbour
- 0.1: go to each of the other 3 states

Reward (depends on next state):

$$R(s') = \begin{cases} 10 & s' = s_4 \\ -1 & \text{otherwise} \end{cases}$$

$$\gamma = 0.9, \quad V(s_4) = 0$$

Iteration 0: Initial values

Initialize:

$$V(s_1) = V(s_2) = V(s_3) = 0, \quad V(s_4) = 0$$

We now compute $V(s_2)$

That is:

$$Q(s_2, \text{Right}) = 0.7(10+0.9\cdot 0)+0.1(-1)+0.1(-1)+0.1(-1) = 6.7$$

Similarly:

$$Q(s_2, \text{Left}) = 0.7(-1) + 0.1(10) + 0.1(-1) + 0.1(-1) = 0.1$$

Action Up / Down: (similar calculation)

$$Q = 0.1$$

Action Right: $Q(s_2, \text{Right}) =$

$0.7(10+0.9\text{V}(\text{s}_4)) + 0.1(1 + 0.9V(s_1)) + 0.1(1 + 0.9V(s_3)) + 0.1(1 + 0.9V(s_2))$

Update $V(s_2)$

Action	$Q(s_2, a)$
Right	6.7
Left	0.1
Up	0.1
Down	6.7

$$V(s_2) = \max_a Q(s_2, a) = 6.7$$

So, from s_2, the best actions: Right or Down

Similarly, can calculate $V(s_1) = 3.22$ *etc.*

Iteration 2: Now γ matters

Recompute $Q(s_2, \text{Right})$ using updated values:

$$= 0.7(10+0.9\cdot 0)+0.1(-1+0.9\cdot 3.22)+0.1(-1+0.9\cdot 0)+0.1(-1+0.9\cdot 6.7)$$

That is:

$$Q(s_2, \text{Right}) = 7 + 0.1(1.898) - 0.1 + 0.1(5.03) \approx 7.5928$$

That is, after the second iteration

$$V(s_2) \text{ increases from } 6.7 \text{ to } 7.59$$

What is happening?

- First iteration: only immediate rewards matter
- Later iterations: future values propagate
- γ controls how strongly this propagation happens

Value iteration = repeatedly improving estimates until convergence

Unrolling Bellman equation

Fix a state s_2 and an action (say, Right).

$$Q(s_2, \text{Right}) = \sum_{s'} P(s'|s_2, \text{Right}) \left[R(s') + \gamma V(s')\right]$$

The recursion comes from $V(s')$. We now expand $V(s')$ step-by-step.

(a) Two-step rollout

Substitute the Bellman equation once:

$$V(s') = \max_{a'} \sum_{s''} P(s''|s', a') \left[R(s'') + \gamma V(s'') \right]$$

$Q(s_2, \text{Right}) =$

$$\sum_{s'} P(s'|s_2, \text{Right}) \Big[R(s') + \gamma \max_{a'} \sum_{s''} P(s''|s', a') \big(R(s'') + \gamma V(s'') \big) \Big]$$

Interpretation:

- reward at step 1
- plus discounted reward at step 2

Final takeaway

- Bellman equation defines value recursively
- $Q(s, a)$ evaluates each action
- $V(s)$ picks the best action
- Repeated updates resolve circular dependencies

Value = best expected future reward

Why value iteration converges

Bellman update:

$$V_{k+1} = T(V_k)$$

Since $\gamma < 1$:

- differences shrink each iteration
- updates converge to a unique fixed point

Value iteration always converges to the optimal value function

Why Learning is Needed

In many real problems:

- transition probabilities are unknown
- rewards are unknown

Reinforcement learning algorithms learn by:

- interacting with the environment
- updating value estimates

Example algorithms:

- Q-learning
- Policy gradients

From Bellman equation to Q-learning

So far we used:

$$Q(s,a) = \mathbb{E}[R(s') + \gamma \max_{a'} Q(s',a')]$$

Problem:

- We need $P(s'|s,a)$ (transition probabilities)
- In real problems, we often do **not know them**

Idea:

Learn $Q(s,a)$ directly from experience

Key idea of Q-learning

Instead of computing expectations, we use samples.

When the agent takes action a in state s and observes:

$$s \to s', \quad \text{reward } r$$

Q-learning update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[R(s') + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$$

Interpretation:

- $Q(s,a)$: current (estimate) of value of taking action a in s
- $R(s')$: immediate reward of taking action a in s
- $\max_{a'} Q(s',a')$: best we can do from next state s'
- $R(s') + \gamma \max_{a'} Q(s',a')$: what $Q(s,a)$ should be
- $\alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$: error in Q-value estimate

Q-learning in the Grid World

s_1	s_2
s_3	s_4

- Goal: s_4 gives reward $+10$
- Other moves: reward -1
- $\gamma = 0.9$
- Learning rate: $\alpha = 0.5$

Initialize:

$$Q(s, a) = 0 \quad \text{for all } s, a$$

Step 1: First experience

Agent starts at s_2, takes action Right.

Transition:

$$s_2 \xrightarrow{\text{Right}} s_4$$

Reward:

$$r = 10$$

Update:

$$Q(s_2, \text{Right}) = 0 + 0.5\big[10 + 0.9 \cdot 0 - 0\big] = 5$$

Step 2: Another experience

Agent at s_1, takes action Right $\rightarrow$ goes to s_2

Reward:

$$r = -1$$

Now:

$$\max_{a'} Q(s_2, a') = 5$$

Update:

$$Q(s_1, \text{Right}) = 0 + 0.5\big[-1 + 0.9 \cdot 5 - 0\big]$$

$$= 0.5(3.5) = 1.75$$

Step 3: Repeat transition $s_2 \rightarrow s_4$

Same transition again:

$$s_2 \xrightarrow{\text{Right}} s_4$$

Update:

$$Q(s_2, \text{Right}) = 5 + 0.5\big[10 - 5\big]$$

$$= 5 + 2.5 = 7.5$$

Value increased

Step 4: Update propagates back

Now revisit $s_1 \to s_2$

$$\max_{a'} Q(s_2, a') = 7.5$$

Update:

$$Q(s_1, \text{Right}) = 1.75 + 0.5\big[-1 + 0.9 \cdot 7.5 - 1.75\big]$$

$$= 1.75 + 0.5(4.75 - 1.75) = 1.75 + 1.5 = 3.25$$

What is happening?

- Rewards are discovered through experience
- Values propagate backward through updates
- Estimates improve over time

Q-learning approximates the Bellman equation using samples

Bellman vs Q-learning (formula comparison)

Bellman equation:

$$Q(s,a) = \sum_{s'} P(s'|s,a) \left[R(s') + \gamma \max_{a'} Q(s',a') \right]$$

Q-learning update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R(s') + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

Key difference:

- Bellman: averages over all possible s'
- Q-learning: uses one observed transition (s, a, r, s')

Does Q-learning converge?

Yes — under some conditions.

Conditions:

- all state-action pairs are explored infinitely often
- learning rate decreases "appropriately"
- we have a finite MDP with bounded rewards

Why it works:

- Bellman operator has a unique fixed point
- Q-learning approximates it using samples
- updates become smaller over time

Q-learning converges to Q^* (satisfies Bellman)

Limitations of Greedy Search

So far, we have learned how to estimate $Q(s, a)$.

But how should the agent **choose actions**?

- If we always pick the best action (greedy choice):
  - we may miss better options
  - We need to explore less promising options
- If we always explore:
  - we never exploit what we have learned

We need a balance between exploration and exploitation

Epsilon-greedy strategy

A simple solution:

$$\text{With probability } 1-\epsilon: \quad \text{choose best action}$$

$$\text{With probability } \epsilon: \quad \text{choose random action}$$

- ϵ is a small number (e.g. 0.1)
- ensures occasional exploration

Mostly exploit, sometimes explore

Example

Suppose at state s:

$$Q(s, \text{Right}) = 6.7, \quad Q(s, \text{Left}) = 0.1$$

With $\epsilon = 0.1$:

- 90%: choose Right (best action)
- 10%: choose randomly (could pick Left)

Even bad actions are tried occasionally

Why exploration is necessary

If we never explore:

- we may get stuck with a suboptimal policy

Example:

- early random experience suggests a poor action is good
- without exploration, we never correct it

Exploration allows learning better strategies

Choosing ϵ

- Large ϵ:
  - more exploration
  - slower convergence
- Small ϵ:
  - more exploitation
  - risk of missing better actions

Common approach:

ϵ decreases over time

Explore early, exploit later

Key takeaway

- ϵ-greedy balances exploration and exploitation
- simple and widely used
- works with Q-learning and SARSA

Learning requires trying new actions

Limitation of tabular methods

So far we used a table:

$$Q(s, a)$$

This works only when:

- number of states is small
- number of actions is small

In real problems:

- states can be images
- states can be continuous
- number of states can be huge

We cannot store $Q(s, a)$ in a table

Key idea of Deep RL

Replace the table with a neural network:

$$Q(s, a) \approx Q_\theta(s, a)$$

- input: state s
- output: value for each action

The network learns to approximate $Q(s, a)$

Deep Q-learning (DQN)

We use the same Q-learning idea:

$$\text{target} = R(s') + \gamma \max_{a'} Q_\theta(s', a')$$

Train the network to match the target:

$$\text{Loss} = \big(Q_\theta(s, a) - \text{target}\big)^2$$

Learning becomes supervised learning

Deep Q-learning: training loop

Initialize:

- neural network $Q_\theta(s, a)$
- replay buffer $\mathcal{D}$

Repeat:

1. Observe current state s
2. Choose action a (using ϵ-greedy)
3. Execute action, observe r, s'
4. Store (s, a, r, s') in buffer $\mathcal{D}$
5. Sample a batch from $\mathcal{D}$
6. Compute targets:

$$y = r + \gamma \max_{a'} Q_\theta(s', a')$$

7. Update network to reduce:

$$(Q_\theta(s, a) - y)^2$$

8. Set $s \leftarrow s'$

How do we compute the target y?

Given (s, a, r, s'):

1. Compute $Q(s', a')$ for all actions
2. Take the maximum:
$$\max_{a'} Q(s', a')$$
3. Compute:
$$y = r + \gamma \max_{a'} Q(s', a')$$

We use the model's current estimate at the next state

Why Deep RL matters

Deep RL enables:

- playing video games from pixels
- robotics control
- autonomous driving

Combining neural networks + RL = powerful AI systems

From value-based to policy-based methods

So far:

$$Q(s, a) \rightarrow \text{choose best action}$$

New idea:

$$\pi_\theta(a \mid s) = \text{probability of action}$$

- directly learn the policy
- no need to compute $Q(s, a)$

Learn actions directly

Example policy

At a state s, suppose:

$$\pi_\theta(\text{Left} \mid s) = 0.3$$

$$\pi_\theta(\text{Right} \mid s) = 0.7$$

The agent:

- chooses Right 70% of the time
- chooses Left 30% of the time

Policy is stochastic

Learning idea

Suppose we take action a and get reward r.

- If reward is high:
  - increase probability of a
- If reward is low:
  - decrease probability of a

Reinforce good actions

Policy gradient update

$$\theta \leftarrow \theta + \alpha\, r\, \nabla_\theta \log \pi_\theta(a \mid s)$$

Interpretation:

- $\nabla_\theta \log \pi_\theta(a \mid s)$:
  - direction to increase probability of action
- multiplied by reward r

Higher reward $\rightarrow$ bigger update

Example: Policy learning in the Grid World

One state s, two actions:

$$a \in \{\text{Left}, \text{Right}\}$$

Policy (softmax):

$$\pi(a) = \frac{e^{\theta_a}}{e^{\theta_L} + e^{\theta_R}}$$

Initial parameters:

$$\theta_L = 0, \quad \theta_R = 0$$

Initial probabilities

$$\pi(\text{Left}) = \pi(\text{Right}) = 0.5$$

The agent picks an action randomly.

Suppose it chooses:

$$a = \text{Right}$$

and receives reward:

$$r = 10$$

Policy gradient update

Update rule:

$$\theta \leftarrow \theta + \alpha\, r\, \nabla_\theta \log \pi(a)$$

Key result (for softmax):

$$\nabla_{\theta_R} \log \pi(\text{Right}) = 1 - \pi(\text{Right})$$

$$\nabla_{\theta_L} \log \pi(\text{Right}) = -\pi(\text{Left})$$

Compute gradients

Current probabilities:

$$\pi(\text{Right}) = 0.5, \quad \pi(\text{Left}) = 0.5$$

$$\nabla_{\theta_R} = 1 - 0.5 = 0.5$$

$$\nabla_{\theta_L} = -0.5$$

Update parameters

Learning rate: $\alpha = 0.1$, reward $r = 10$

$$\theta_R = 0 + 0.1 \cdot 10 \cdot 0.5 = 0.5$$

$$\theta_L = 0 + 0.1 \cdot 10 \cdot (-0.5) = -0.5$$

New probabilities

$$\pi(\text{Right}) = \frac{e^{0.5}}{e^{0.5} + e^{-0.5}} \approx 0.73$$

$$\pi(\text{Left}) \approx 0.27$$

The agent is now more likely to choose Right

Comparison with Q-learning

Q-learning	Policy gradient
Learns $Q(s, a)$	Learns $\pi(a\|s)$
Needs max	No max needed
Indirect policy	Direct policy

Big picture

- Q-learning $\rightarrow$ learn values
- Policy gradients $\rightarrow$ learn policy
- Actor–Critic $\rightarrow$ combine both

Modern RL uses Actor–Critic ideas

Famous successes of reward-based learning

1. **TD-Gammon (Tesauro, 1992–1995)**
   - Learned to play backgammon at world-class level
   - Used temporal-difference (TD) learning with neural networks
2. **Atari Games (DeepMind, 2015)**
   - Deep Q-Networks (DQN) achieved human-level performance
   - Learned directly from raw pixels
3. **AlphaGo (DeepMind, 2016)**
   - Defeated world champion Lee Sedol
   - Combined RL with search and deep learning
4. **Robotics and Control**
   - Learning to grasp objects and control movement
   - Applications in autonomous systems and industry
5. **RLHF**
   - Aligns AI behavior using human feedback
   - Learns reward functions from human preferences

Beyond MDPs

So far we assumed the agent sees the true state.

In reality:

- sensors are noisy
- the environment may be partially observable

This leads to:

Partially Observable MDPs (POMDPs)
(not covered here)